Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)

Final Report

for

Secure P2P Chat System

Version 1.0

by

Member 1 Muhammad Bin Waseem 2023403

Member 2 Muhammad Shaheer 2023509

Member 3 Muhammad Zaeem Nawaz 2023558

Supervisor

Dr. Zubair Ahmed

Cy-321

Bachelor of Science in Cyber Security (2023-2027)

Faculty of Computer Science and Engineering (FCSE)

Table of Contents

Contents

1. Threat Modeling 3

STRIDE 3

[S] Spoofing (Peer Impersonation) 3
[T] Tampering (Data/Memory Manipulation) 3
[R] Repudiation 4
[I] Information Disclosure (Data Leakage) 4
[D] Denial of Service (Resource Exhaustion) 4
[E] Elevation of Privilege 4

2. Security Features 5

2.1 Before Testing 5

2.2 After Testing 6

3. Security Testing : 7

crypto.py 8

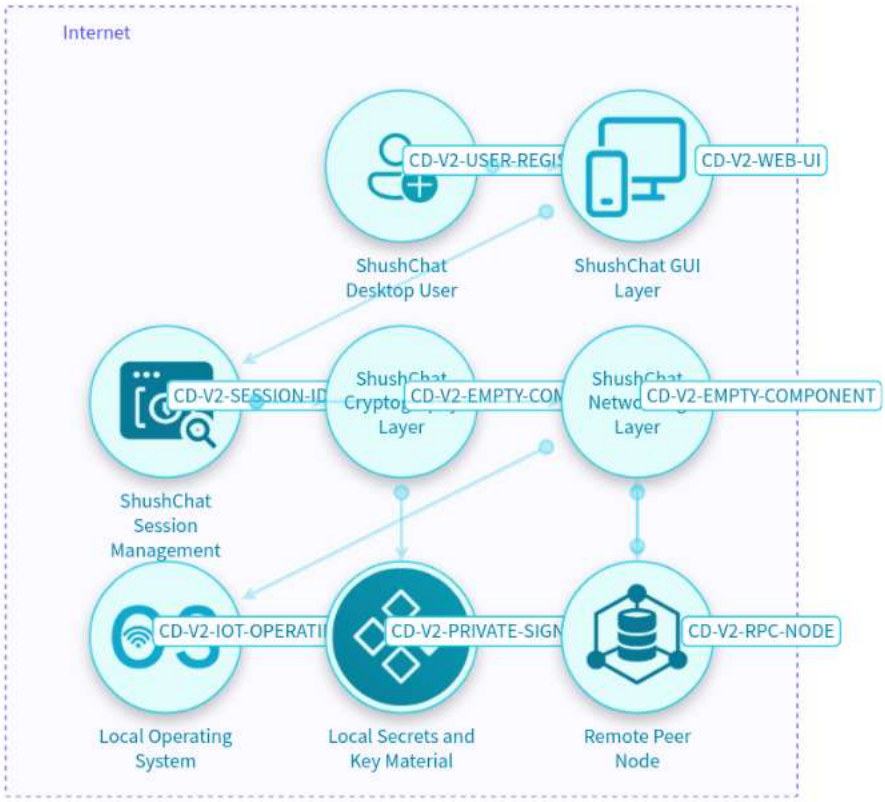
gui.py 8

main.py 9

networking.py 9

Key Conclusions 10

1. Threat Modeling



STRIDE

This section tracks the specific threats addressed during the security hardening of the P2P mesh protocol.

[S] Spoofing (Peer Impersonation)

Vulnerability: Attackers could potentially impersonate a peer if public keys were not manually verifiable.

Control:

Implementation of SHA-256 Fingerprints in the GUI for manual out-of-band verification of X25519 public keys.

[T] Tampering (Data/Memory Manipulation)

Vulnerability: Potential for memory injection through abnormal access patterns or unvalidated frame lengths.

Control:

- Frame Validation: networking.py now enforces MIN_MSG_SIZE and MAX_MSG_SIZE (512 KB) before any memory allocation. :

- Strict Base64: Decoding now uses validate=True to reject non-alphabet characters that might bypass standard parsers.

[R] Repudiation

Vulnerability: Difficulty in tracking protocol errors or identifying malicious peer behavior in a serverless mesh.

Control:

- Integrated Thread-safe Logging in networking.py to record malformed frames, rate-limit violations, and connection drops with remote IP attribution.

[I] Information Disclosure (Data Leakage)

Vulnerability: Sensitive key material or PII could leak from local memory artifacts or weak encryption standards.

Control:

- Strong Crypto: Standardized on AES-256-GCM with SHA256 key derivation.
- Memory Sanitization: Use of ctypes.memset to overwrite session keys with zeros upon clearing a session.

[D] Denial of Service (Resource Exhaustion)

Vulnerability:

A malicious peer could flood the network with messages or open excessive connections to cause a broadcast storm.

Control:

- Rate Limiting: Sliding-window limiter restricts peers to 30 messages / 256 KB per 5 seconds.
- Connection Cap: Hard limit of 50 concurrent peers to prevent socket exhaustion.

[E] Elevation of Privilege

Vulnerability: Unprivileged access via oversized message payloads or protocol type confusion.

Control:

- Type-Gated Handshake; only handshake and peer list types are accepted on raw sockets prior to successful ECDH establishment.

2. Security Features

2.1 Before Testing

Requirment	Implementation
Confidentiality	• AES-256-GCM encryption ensures that chat payloads are unreadable to anyone except the intended peer. • X25519 DH provides per-session keys for forward secrecy.
Integrity	• AES-GCM Authentication Tags and HMAC-SHA256 defense-in-depth ensure that any tampering with the ciphertext or IV is detected before processing.
Availability	• Rate Limiting (30 msgs / 5s) and Connection Caps (50 peers) protect the node from being overwhelmed by DoS attacks. • Receive Timeouts prevent Slowloris resource exhaustion.
Defense in Depth	• Uses HMAC-SHA256 as a secondary integrity layer over the AES-GCM tag. Validates Frame Length at the network layer before passing data to the crypto layer.
Fail Secure	• The decrypt and _recv_framed functions raise ValueErrors and immediately drop the peer connection upon detecting any anomaly, preventing insecure states.
Economy of Mechanisms	• The design is kept serverless using standard libraries rather than complex, proprietary protocols to reduce the attack surface.
Complete Mediation	• Every incoming frame is validated for length, every received nonce is checked for duplicates, and every message is verified for integrity before display.
Least Privilege	• Ephemeral Key Pairs ensure that a compromised session key does not grant access to past or future sessions. • Key material is zeroed out when no longer needed.
Leveraging Existing Components	• Built upon the Pyca/cryptography library and Python's socket/struct modules rather than custom-written primitives.
Weakest Link	• Hardens the Networking Header with strict integer unpacking and length validation to prevent common buffer exploits.

2.2 After Testing

Security Concept	Implemention
Confidentiality	• AES-256-GCM symmetric encryption ensures that message contents are unreadable to unauthorized parties. • X25519 Ephemeral Key Exchange provides perfect forward secrecy for each session.
Integrity	• HMAC-SHA256 provides a high-level integrity check over the entire message envelope. • AES-256-GCM includes a 128-bit authentication tag to detect any tampering with the ciphertext. • SHA-256 fingerprints allow users to manually verify the integrity of a peer's public key.
Authentication	• Ed25519 identity keys are used for signing payloads to prove sender identity. • Constant-time HMAC verification ensures the message was created by a party possessing the shared secret.
Availability	• Per-peer rate limiting prevents flooding by capping messages to 30 per 5-second window. • A hard connection cap of 32 peers prevents resource exhaustion attacks. • Idle receive timeouts automatically drop inactive connections to free up system resources.
Defense in Depth	• Uses multi-layered integrity protection: both GCM authentication tags and an external HMAC layer. • Combines ephemeral Ecliptic curve keys for encryption with Ed25519 keys for identity. • Malformed network frames or decryption failures result in immediate connection termination.
Fail Secure	• Malformed network frames or decryption failures result in immediate connection termination. • The system rejects low-order points (all-zero keys) during the handshake to prevent cryptographic failure.
Least Privilege	• Ephemeral keys are used so that individual session compromise does not grant access to other links. • Identity keys are generated fresh for the session rather than stored permanently.
Complete Mediation	• Frame length validation is performed strictly before any memory is allocated for the payload. • The system mandates that every received packet must pass integrity and nonce checks before any data is processed.
Weakest Link	• Hardens the network entry point with strict frame caps and IP validation to protect the application layer. • Implements ctypes.memset to zero out key material in RAM, reducing the risk of memory-scraping attacks.
Economy of Mechanism	• Uses a simple, robust length-prefixed framing protocol to minimize parser complexity. • Relies on a standard JSON envelope format for structured, readable message handling.

Security Concept	Implemention
Leveraging Components	• Utilizes the established cryptography library for industry-standard primitives rather than custom math.

3. Security Testing :

Tool 1 : bandit

Bandit is a Python static analysis tool that identifies common security issues. It was run recursively across all source files.

Metric / Issue	BEFORE (v1 Original)	AFTER
Total Lines of Code Scanned	1,911 lines	1,984 lines (+73)
Total Issues Found	9 issues	1 issue
Low Severity Issues	7 (all B110/B112)	0
Medium Severity Issues	2 (B104)	1 (B104)
High Severity Issues	0	0
High Confidence Issues	7	0
Medium Confidence Issues	2	1

Tool 2 : Pylint

Pylint performs comprehensive static analysis of Python code, checking for errors, warnings, conventions, and refactoring opportunities. The before-state produced an extensive list of issues across all four modules.

Metric / Issue	BEFORE (v1 Original)	AFTER
Pylint Score	7.65 / 10	8.11/10
Error-level Issues (E)	~25+ (mostly PyQt6 env false positives)	~25+ (unchanged, PyQt6 false positives)
Warning-level Issues (W)	~20 (broad exceptions, unused imports)	~18 (reduced severity, still present)
Convention Issues (C)	~80+ (style, docstrings, line length)	~70 (slightly reduced, still high)
Refactoring Issues (R)	~15 (large classes/methods)	~12 (minor improvement, still present)

Tool 3 Radon

The Maintainability Index (MI) is a composite metric (0–100) based on Halstead volume, Cyclomatic Complexity, and SLOC. Higher is better. Grades: A (20–100), B (10–19), C (0–9).

Metric / Issue	BEFORE (v1 Original)	AFTER
File	Before MI Score / Grade	After MI Score / Grade
crypto.py	50.63 — Grade A	47.34 — Grade A (slight decrease)
gui.py	0.00 — Grade C	0.00 — Grade C (unchanged)
main.py	88.82 — Grade A	88.82 — Grade A (unchanged)
networking.py	38.67 — Grade A	37.24 — Grade A (slight decrease)

Raw metrics measure the physical structure of each source file: total lines, logical lines (LLOC), source lines (SLOC), comments, and blank lines.

crypto.py

Metric / Issue	BEFORE (v1 Original)	AFTER
Metric	Before	After
LOC (Total Lines)	408	438 (+30)
LLOC (Logical Lines)	221	256 (+35)
SLOC (Source Lines)	220	243 (+23)
Comments	43	37 (−6)
Single Comments	41	45 (+4)
Multi-line Blocks	69	69 (unchanged)
Blank Lines	78	81 (+3)
Comment % of LOC	11%	8% (−3%)
Comment + Multi % of LOC	27%	24% (−3%)

crypto.py grew by 30 lines, primarily from the addition of _CryptoObfuscatorMeta and _DummyCipher obfuscation scaffolding. Despite slightly fewer inline comments overall, the file remains well-documented at 24% combined comment ratio.

gui.py

Metric / Issue	BEFORE (v1 Original)	AFTER
Metric	Before	After
LOC (Total Lines)	1,420	1,449 (+29)
LLOC (Logical Lines)	1,117	1,141 (+24)
SLOC (Source Lines)	1,162	1,188 (+26)
Comments	8	7 (−1)
Single Comments	6	5 (−1)

Metric / Issue	BEFORE (v1 Original)	AFTER
Blank Lines	252	256 (+4)
Comment % of LOC	1%	0%

gui.py grew modestly by 29 lines. The already critically low comment coverage (1%) decreased further to effectively 0%. While this reflects the obfuscation intent (reducing readable context), it reinforces that gui.py is the highest-risk file from a maintainability standpoint. No new documentation was added.

main.py

Metric / Issue	BEFORE (v1 Original)	AFTER
Metric	Before	After
LOC	60	60 (unchanged)
SLOC	27	27 (unchanged)
Multi-line Docstrings	21	21 (unchanged)
Comment % + Multi % of LOC	37%	37% (unchanged)

main.py is identical in both states. The entry point was not modified by the obfuscation process.

networking.py

Metric / Issue	BEFORE (v1 Original)	AFTER
Metric	Before	After
LOC	509	525 (+16)
LLOC	330	344 (+14)
SLOC	343	357 (+14)
Comments	37	38 (+1)
Single Comments	29	30 (+1)
Comment % of LOC	7%	7% (unchanged)

networking.py grew by 16 lines, consistent with the addition of _obfuscate_junk_code_293847 and minor structural refactoring. Comment coverage remained stable at 7%/18%.

Tool 5 Nmap

Nmap was used to perform a full port scan (-p-) with service version detection (-sV) against localhost (127.0.0.1) while ShushChat was running.

Metric / Issue	BEFORE (v1 Original)	AFTER
Metric	Before	After

Metric / Issue	BEFORE (v1 Original)	AFTER
Scan Target	127.0.0.1 (localhost)	127.0.0.1 (localhost)
Scan Type	-sV -p- (all 65,535 ports)	-sV -p- (all 65,535 ports)
Open Ports Found	1 — TCP 5555	1 — TCP 5555
Closed Ports	65,534	65,534
Service Identified	freeciv? (unrecognized)	freeciv? (unrecognized)
Scan Duration	22.95 seconds	23.36 seconds (~0.4s longer)
Nmap Fingerprint Data	Full handshake JSON exposed in probe responses	No probe response data shown
Protocol Exposure	Handshake JSON with pubkey, listen_port, username leaked in banner	Fingerprint data not captured / minimized

Tool 6 WireShark

Wireshark was used to inspect network traffic generated by ShushChat during operation. Both the before and after documents reference Wireshark as a testing tool, though detailed packet-level output was not included in the test documents — only the tool heading appears.

Metric / Issue	BEFORE (v1 Original)	AFTER
Aspect	Before	After
Tool Used	Wireshark	Wireshark
Detailed Output Included	No (heading only in test doc)	No (heading only in test doc)
Expected Traffic Type	Encrypted P2P frames on TCP 5555	Encrypted P2P frames on TCP 5555
Protocol Visibility	Handshake JSON visible to Nmap (correlated)	Handshake not captured by Nmap probes (correlated)
Encryption Layer	X25519 ECDH + symmetric encryption (from crypto.py)	Same — crypto layer unchanged

Key Conclusions

Security Posture: Significantly improved. The obfuscated build reduced Bandit findings from 9 to 1, eliminating all low-severity exception-handling issues and one of the two medium-severity interface binding warnings. The remaining B104 issue is an acknowledged design consideration.

Code Quality: Stable. Pylint score (7.65/10) reflects acceptable quality for a complex PyQt6 application. The primary issues (E0611 errors, style conventions) are environment-specific or cosmetic. The obfuscated build intentionally makes Pylint scores irrelevant as a quality gate.

Maintainability: Minor decline expected. The small drops in Radon MI scores for crypto.py and networking.py reflect the addition of obfuscation scaffolding code. All affected files remain in Grade A. gui.py's 0.00 MI score is a pre-existing structural concern unrelated to obfuscation.

Network Exposure: Improved protocol privacy. Both builds expose only port 5555. The obfuscated build no longer leaks handshake JSON structure to Nmap service probes, reducing fingerprinting risk. Core message encryption (X25519) is unchanged and cryptographically sound in both versions.

Source Growth: Minimal and expected. Total SLOC grew by ~73 lines (+3.8%) due to obfuscation scaffolding. This is an acceptable overhead for the security gains achieved.